SOEN 363
Data Systems for Software Engineers

Project 1

20/10/2026



Mohammad Alhaji (40264810)

Juan Manuel Alvarez (40374997)

Zachary Demnati (40340103)

Laurent Lao (40020483)

Freddy Mvemba (25745616)

Contents

Introduction 2
System Overview 2
Entity-Relationship Diagram 3
Data Population 7
Reproduction Steps 8
Conclusion 8
Appendix A — Individual Query Screenshots 9
..... 13
Table of Figures 29

Introduction

For this project, our team was tasked with building a Hospital Management System (HMS) database from the ground up. We split the work into three main phases: first, designing an Entity-Relationship (ER) model that maps out how hospital data connects; second, translating that design into actual SQL code using PostgreSQL; and third, populating the database with realistic sample data. To make the data useful for testing, we wrote a Python script to automatically generate a large, consistent dataset. The final deliverables for this phase include the ER diagram, the table schemas, the data population scripts, and a set of 20 analytical SQL queries that we wrote to test the database's capabilities.

System Overview

Our database covers the main operations you would expect in a hospital setting. It tracks patients from the moment they are admitted, including their initial triage vitals, any diagnoses they receive (using ICD-9 codes), and their discharge records. It also handles the staff side of things, linking physicians and nurses to specific admissions, notes, and procedures. We also included tables for more complex workflows, such as radiology orders, ICU stays, and surgical procedures.

When designing the schema, our main priority was keeping the data clean. We normalized the tables to Third Normal Form (3NF) to prevent redundant data, and we set up foreign keys to make sure that, for example, a diagnosis couldn't be entered for an admission that doesn't exist. We also made sure our data generation script produced a consistent dataset so that anyone could re-run it and get the exact same results.

Entity-Relationship Diagram

Figure 1:ER Diagram

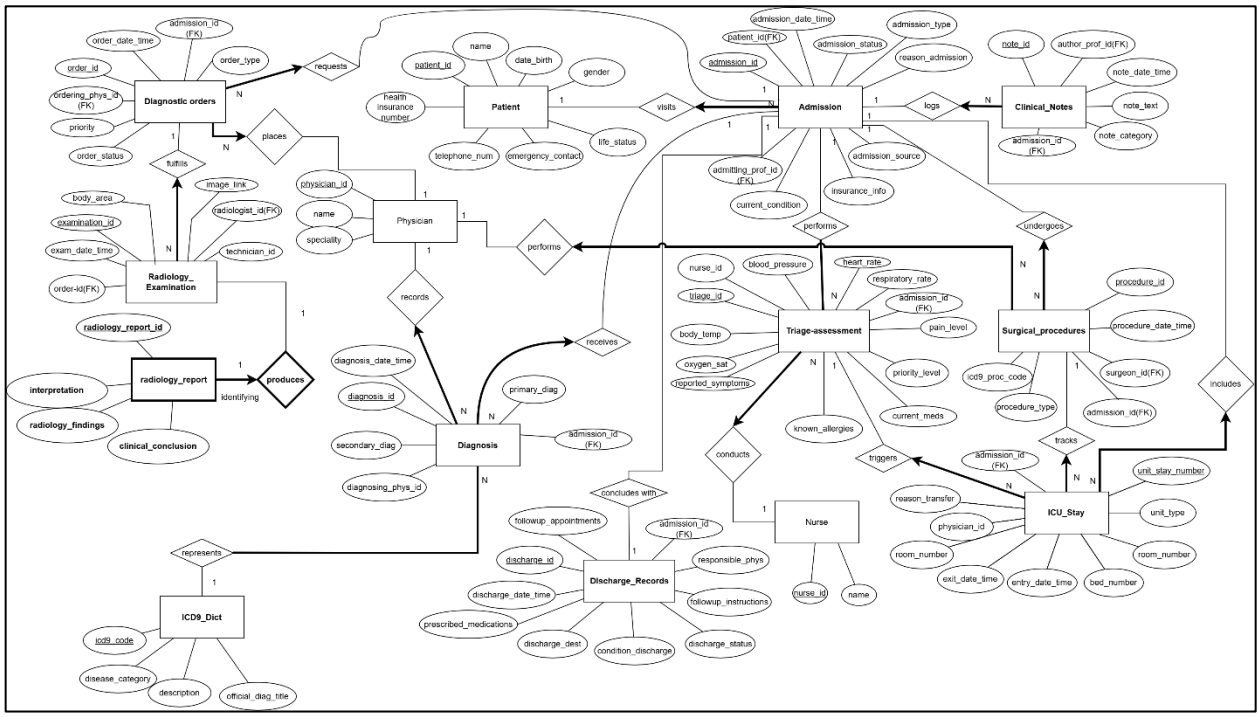


Figure 1 shows our ER diagram. We modelled the patient journey as a chain: a patient has many admissions; each admission is linked to one triage_assessment, one or more diagnosis records (with exactly one marked as primary), optional icu_stay and surgical_procedure records, and one discharge_record. Staff are modelled separately: physician and nurse connect to admissions through the tables where they act (e.g., admitting_prof_id on admission, nurse_id on triage). Diagnostic work follows a strict path — diagnostic_order → radiology_examination → radiology_report — because one order can produce one exam, and each exam has one report. We made icu_stay a separate entity (rather than a column on admission) because a patient can move between units (ICU, CCU, MICU, SICU) during a single stay, and Q15/Q16 requires counting those transfers. Cardinality and participation constraints are marked on the diagram: for example, a patient may have zero admissions, but every admission must belong to exactly one patient.

Relational Schema

The database schema was designed according to Third Normal Form (3NF). Each table represents a specific entity in the hospital system, which helps reduce data duplication and maintain consistency.

Primary keys uniquely identify records in each table, while foreign keys define relationships between tables. For example, each admission is linked to a patient, and diagnoses are linked to both admissions and physicians.

The database consists of 14 tables that work together to model hospital operations and support the required project queries.

Below is the formal relational schema mapping for the 14 operational tables:

- icd9_dict (icd9_code, disease_category, description, official_diag_title)
- physician (physician_id, name, specialization)
- nurse (nurse_id, name)
- patient (patient_id, name, date_birth, gender, address, telephone_number, emergency_contact, life_status)
- admission (admission_id, patient_id, admission_date_time, admission_type, reason_admission, admission_source, insurance_info, current_condition, admitting_prof_id)
    - Foreign Keys: patient_id references patient(patient_id), admitting_prof_id references physician(physician_id)
- triage_assessment (triage_id, *admission_id*, *nurse_id*, blood_pressure, heart_rate, respiratory_rate, body_temp, oxygen_sat, pain_level, priority_level, reported_symptoms, known_allergies, current_meds)
    - Foreign Key*:* admission_id references admission(admission_id), nurse_id references nurse(nurse_id)
- diagnosis (diagnosis_id, admission_id, *icd9_code*, diagnosing_phys_id, diagnosis_date_time, primary_diag, secondary_diag)
    - Foreign Keys*:* admission_id references admission(admission_id), icd9_code references icd9_dict(icd9_code), diagnosing_phys_id references physician(physician_id)
- icu_stay (unit_stay_number, admission_id, unit_type, room_number, bed_number, entry_date_time, exit_date_time, physician_id, reason_transfer)
    - Foreign Keys*:* admission_id references admission(admission_id), physician_id references physician(physician_id)
- clinical_note (note_id, admission_id, author_prof_id, note_date_time, note_text, note_category)
    - *Foreign Keys:* admission_id references admission(admission_id)

- diagnostic_order (order_id, *admission_id*, *ordering_phys_id*, order_date_time, order_type, order_status)
    - Foreign Keys*:* admission_id references admission(admission_id), ordering_phys_id references physician(physician_id)
- radiology_examination (examination_id, *order_id*, exam_date_time, modality, technician_id, *radiologist_id*)
    - Foreign Keys*:* order_id references diagnostic_order(order_id), radiologist_id references physician(physician_id)
- surgical_procedure (procedure_id, *admission_id*, *surgeon_id*, *icd9_proc_code*, procedure_date_time, phone_number)
    - Foreign Keys*:* admission_id references admission(admission_id), surgeon_id references physician(physician_id)
- discharge_record (discharge_id, *admission_id*, discharge_date_time, discharge_dest, condition_discharge, discharge_status, followup_instructions, *responsible_phys*)
    - Foreign Keys*:* admission_id references admission(admission_id), responsible_phys references physician(physician_id)

Data Population

The database was populated using a Python script that generated synthetic hospital data. No real patient information was used in this project.

The generated dataset contains records for patients, physicians, nurses, admissions, diagnoses, clinical notes, ICU stays, radiology examinations, and surgical procedures. The goal was to create enough data to properly test the database structure and all required SQL queries.

The script produces an SQL file containing INSERT statements that can be executed directly in PostgreSQL to populate the database.

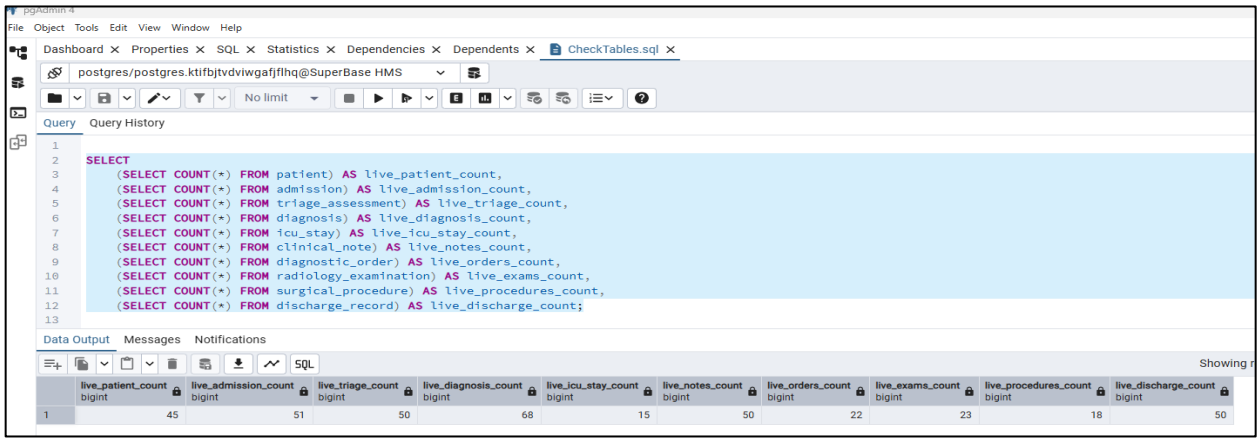
A detailed description of the dataset generation process, AI usage, prompts, transformation steps, validation procedures, and import methodology is provided in Comprehensive_Data_Population_Report. All database records were synthetically generated for educational purposes and no real patient data was used.

The generate_data.py script creates *populate_all_hospital.sql*, which contains the sample-data INSERT statements.

The generated dataset includes:

- 4 ICD-9 codes, 15 physicians, 10 nurses, and 45 patients.
- 51 admissions, 50 triage assessments, 68 diagnoses, 15 ICU stays, and 50 clinical notes.
- 22 diagnostic orders, 23 radiology examinations, and radiology reports.
- 50 discharge records and 18 surgical procedures.

Figure 2: Live statics of our database



See Appendix A — Individual Query Screenshots— Individual Query Screenshots for a list of Queries.

Reproduction Steps

1. Create an empty PostgreSQL database named *Superbase HMS*.
2. Run *TablesCreation.sql* to create the database schema.
3. Run *populate_all_hospital.sql* to load the sample data.
4. Run *CheckTables.sql* and verify the expected row counts.
5. Run Q1 through Q20 from *FirstPart.sql*.

Conclusion

Looking back, we successfully completed all the main goals for Phase 1. We ended up with a fully normalized, 14-table database in PostgreSQL that handles everything from patient admissions to surgical procedures. By sticking to 3NF, we avoided data duplication and used foreign keys to keep the relationships between tables strict and reliable.

Generating the test data was one of the more challenging parts, but the Python script worked well in the end, giving us a consistent dataset of 45 patients and 51 admissions to work with. Finally, writing the 20 queries was a good way to see if our design actually held up. By using complex joins and subqueries to answer realistic questions—like tracking ICU transfers or finding patients with multiple diagnoses—we were able to confirm that the database is structured correctly and can handle real analytical tasks.

Appendix A — Individual Query Screenshots

Q1: [Zachary*]*

List the names and admission dates of all patients who were classified as high-priority emergency patients during triage.

Query:

Figure 3: Query 1

```
SELECT p.name, a.admission_date_time
FROM admission a
JOIN patient p ON a.patient_id = p.patient_id
JOIN triage_assessment t ON a.admission_id = t.admission_id
WHERE t.triage_priority_level = 'High-Priority Emergency';
```

Output:

Figure 4: Output Query 1

Query Query History

Data Output Messages Notifications

SQL

	name character varying (100)	admission_date_time timestamp without time zone
1	Amelie Taylor	2025-12-30 18:48:47
2	Michael Smith	2026-05-19 05:03:47
3	Jessica Tremblay	2026-09-23 06:30:47
4	Rachel Smith	2026-02-08 18:20:47
5	Daniel Charette	2026-07-15 10:01:47
6	John Tremblay	2026-03-02 19:32:47
7	Michael Taylor	2026-04-05 13:48:47
8	Amelie Gagnon	2026-09-05 23:20:47
9	Michael Roy	2026-05-23 22:30:47
10	Robert Charette	2026-05-20 02:33:47
11	Sarah Smith	2026-09-09 12:34:47
12	Robert Miller	2026-08-03 02:30:47
13	Sarah Smith	2026-01-30 02:05:47
14	Daniel Tremblay	2026-07-06 13:56:47
15	Michael Roy	2026-06-12 21:21:47
16	Amelie Tremblay	2025-12-23 08:09:47
17	David Williams	2026-09-26 17:17:47
18	Robert Jones	2026-03-01 22:01:47
19	Jessica Taylor	2026-03-29 14:24:47
20	Daniel Charette	2026-08-14 00:35:47
21	Robert Miller	2026-09-10 22:02:47
22	David Williams	2026-07-28 10:49:47
23	Amelie Gagnon	2026-03-07 10:00:47
24	Amelie Tremblay	2026-08-28 02:27:47
25	Sarah Smith	2026-09-16 21:46:47
26	Emily Jones	2026-02-23 04:18:47
27	Robert Miller	2026-08-21 11:15:47

Logic: The logic of this query is we first select the name and admission date time which we want to show. They are from 2 different tables, so we need to create patient p and admission a. We use from to indicate that we are selecting from that table and then join to find the patient that has

the same id as the admission that we are selecting. We then select these patients and use them to find the names. Then, same logic but for the triage assessment. We find the triage assessment that has the same id as the admission/patient and then, as a condition we are selecting only those that have their triage priority level to high-priority emergency. We are putting the triage in SELECT since we only want to show the name and admission date/time.

Q2: [Zachary*]*

Find all patients who had more than three hospital admissions. Display the patient identifier, patient name, and total number of admissions.

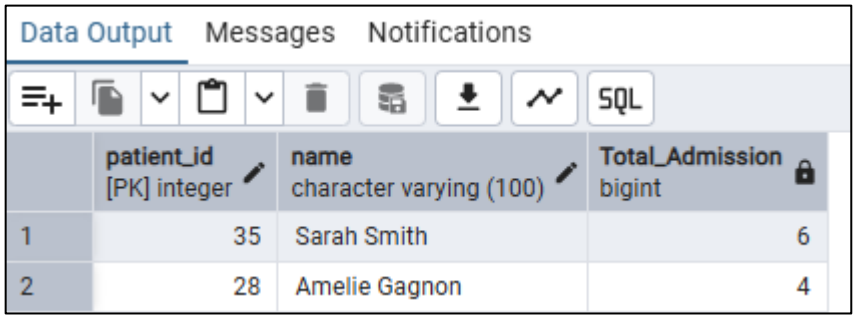
Query:

Figure 5: Query 2

```
SELECT p.patient_id, p.name, COUNT(a.admission_id) AS "Total_Admission"
FROM patient p
JOIN admission a ON a.patient_id = p.patient_id
GROUP BY p.patient_id, p.name
HAVING COUNT(a.admission_id) > 3;
```

Output:

Figure 6: Output Query 2



	patient_id [PK] integer	name character varying (100)	Total_Admission bigint
1	35	Sarah Smith	6
2	28	Amelie Gagnon	4

Logic: The logic of this query is that we first select the patient id and the name of a patient p. Then we want to calculate the total number of admissions. For that we use count on the admission id from the patient p and we display is as total_admissions. Then we use join to only show the admission where the patient id's are the same. And finally, we use group to merge all the rows with the same patient id into one row that displays the total number of admissions. The having will then be the condition for the query. (more than 3 admissions)

Q3: [Zachary*]*

Retrieve the names, admission dates, and discharge dates of patients who were discharged without undergoing any surgical procedure during admission.

Query:

Figure 7: Query 3

```
SELECT p.name, a.admission_date_time, d.discharge_date_time
FROM patient p
JOIN admission a ON p.patient_id = a.patient_id
JOIN discharge_record d ON a.admission_id = d.admission_id
WHERE NOT EXISTS(
    SELECT 1
    FROM surgical_procedure sp
    WHERE sp.admission_id = a.admission_id
)
ORDER BY p.name;
```

Output:

Figure 8: Output Query 3

Data Output | Messages | Notifications

	name character varying (100)	admission_date_time timestamp without time zone	discharge_date_time timestamp without time zone
1	Amelie Gagnon	2026-04-28 21:13:47	2026-04-30 21:13:47
2	Amelie Gagnon	2026-03-07 10:00:47	2026-03-14 10:00:47
3	Amelie Tremblay	2025-12-23 08:09:47	2026-01-02 08:09:47
4	Amelie Tremblay	2026-08-28 02:27:47	2026-08-30 02:27:47
5	Daniel Charette	2025-12-26 18:34:47	2025-12-30 18:34:47
6	Daniel Charette	2026-08-14 00:35:47	2026-08-18 00:35:47
7	Daniel Charette	2026-07-15 10:01:47	2026-07-25 10:01:47
8	Daniel Gagnon	2026-09-28 13:51:47	2026-09-29 13:51:47
9	Daniel Tremblay	2026-07-06 13:56:47	2026-07-08 13:56:47
10	David Charette	2026-01-10 16:26:47	2026-01-21 16:26:47
11	David Williams	2026-07-28 10:49:47	2026-08-03 10:49:47
12	Emily Jones	2026-08-21 09:47:47	2026-09-02 09:47:47
13	Emily Jones	2026-08-16 21:00:47	2026-08-30 21:00:47
14	Jessica Taylor	2026-03-29 14:24:47	2026-04-06 14:24:47
15	Jessica Tremblay	2026-09-23 06:30:47	2026-10-03 06:30:47
16	John Tremblay	2026-03-02 19:32:47	2026-03-08 19:32:47
17	John Tremblay	2026-02-15 14:16:47	2026-02-23 14:16:47
18	Michael Roy	2026-05-22 08:13:47	2026-05-26 08:13:47
19	Michael Roy	2026-05-23 22:30:47	2026-06-03 22:30:47
20	Michael Roy	2026-06-12 21:21:47	2026-06-23 21:21:47
21	Michael Smith	2026-05-19 05:03:47	2026-05-31 05:03:47
22	Michael Taylor	2026-04-05 13:48:47	2026-04-06 13:48:47
23	Michael Taylor	2026-09-13 11:37:47	2026-09-23 11:37:47
24	Rachel Smith	2026-02-08 18:20:47	2026-02-17 18:20:47
25	Rachel Williams	2026-08-09 16:42:47	2026-08-17 16:42:47
26	Robert Charette	2025-12-13 21:36:47	2025-12-27 21:36:47
27	Robert Charette	2026-05-20 02:33:47	2026-05-28 02:33:47
28	Robert Jones	2026-03-01 22:01:47	2026-03-09 22:01:47
29	Sarah Charette	2025-12-20 21:42:47	2025-12-22 21:42:47
30	Sarah Miller	2025-12-20 21:18:47	2026-01-01 21:18:47

Logic: The main logic of this query is that we need to find what link exists between a patient, the admission and the surgical procedure. Between the patient and the admission, it is the patient id, we can then use that to find a patient with certain condition from the admission and patient table. The surgical procedure table is also based on the admission id, so we can do the same thing using join. The final step is to check if the discharge date time is not null because we want patients who were discharged only.

Q4: [Zachary*]*

List all patients who had both a radiology examination and a surgical procedure during the same admission.

Query:

Figure 9: Query 4

```
SELECT DISTINCT p.name
FROM patient p
JOIN admission a ON a.patient_id = p.patient_id
JOIN diagnostic_order d ON d.admission_id = a.admission_id
JOIN radiology_examination r ON r.order_id = d.order_id
JOIN surgical_procedure s ON s.admission_id = a.admission_id;
```

Output:

Figure 10: Output Query 4

Data Output | Messages | Notificati

	name character varying (100)
1	Robert Miller
2	Amelie Tremblay
3	Emily Jones
4	Sarah Smith
5	David Williams
6	Emily Roy

***Logic**: First, we need to select distinct since a single patient can have multiple admissions that satisfy the requirements, so to prevent that, we use the distinct keyword. After that, like in query 4, we need to find a link between every table. For patients and admissions, it's the patient id. After that, for the diagnostic order, its the admissions id. Th*

Q5: [Juan*]*

Find all ICU or CCU unit stays that lasted more than seven days. Display the patient name, unit type, unit entry time, unit exit time, and calculated length of stay

Figure 11: Query 5

```sql
SELECT
    p.name,
    i.unit_type,
    i.entry_date_time,
    i.exit_date_time,
    (i.exit_date_time - i.entry_date_time) AS length_of_stay
FROM icu_stay i
JOIN admission a ON i.admission_id = a.admission_id
JOIN patient p ON a.patient_id = p.patient_id
WHERE i.unit_type IN ('ICU', 'CCU')
  AND i.exit_date_time IS NOT NULL
  AND (i.exit_date_time - i.entry_date_time) > INTERVAL '7 days';
```

Output:

Figure 12: Output Query 5

name character varying (100)	unit_type character varying (50)	entry_date_time timestamp without time zone	exit_date_time timestamp without time zone	length_of_stay interval

Logic: Result: 0 rows. The query correctly joins icu_stay with admission and then patient, restricts to ICU/CCU units, excludes in-progress stays (exit_date_time IS NOT NULL), and only keeps stays longer than 7 days. In the current database no ICU or CCU stay exceeds 7 days.

Q6: [JuanMa*]*

Count the total number of hospital admissions for each patient. Include patients who have no recorded admissions.

Figure 13: Query 6

```
SELECT
    p.patient_id,
    p.name,
    COUNT(a.admission_id) AS total_admissions
FROM patient p
LEFT JOIN admission a ON a.patient_id = p.patient_id
GROUP BY p.patient_id, p.name;
```

Figure 14: Output Query 6

	patient_id [PK] integer	name character varying (100)	total_admissions bigint
1	42	Marc-Andre Tremblay	0
2	29	Daniel Charette	0
3	4	Rachel Williams	2
4	34	Daniel Tremblay	1
5	41	Sarah Tola	1
6	40	Daniel Gagnon	0
7	43	Chloe Boucher	0
8	32	Michael Smith	1
9	7	David Miller	0
10	10	Robert Miller	3
11	9	Jessica Tremblay	1
12	35	Sarah Smith	6
13	45	Elena Petrova	0
14	38	John Williams	0
15	15	Amelie Tremblay	3
16	6	Emily Roy	1
17	12	Sarah Charette	1
18	26	Sarah Miller	2
19	39	Daniel Charette	3
20	24	Emily Jones	3
21	19	Daniel Smith	0
22	25	Daniel Miller	0
23	36	John Tremblay	2
24	31	Robert Jones	1
25	30	Michael Roy	3
26	21	Jessica Taylor	1
27	14	David Williams	2
28	3	Daniel Gagnon	0
29	17	Jessica Gagnon	0
30	22	David Charette	1
31	20	Robert Charette	2
32	37	Amelie Taylor	2
33	28	Amelie Gagnon	4
34	33	Emily Jones	1
35	13	Michael Taylor	2
36	1	Amelie Roy	0
37	5	Emily Taylor	0
38	18	David Gagnon	0
39	2	Robert Gagnon	0
40	16	Daniel Gagnon	1
41	27	Rachel Smith	1
42	23	David Jones	0
43	44	Jean-Francois Roy	0
44	8	Sarah Taylor	0
45	11	Jessica Tremblay	0

Logic: uses a LEFT JOIN from patient to admission so every patient is kept even with no admissions, then groups by patient and uses COUNT(admission_id) to total each one's admissions. Since COUNT ignores nulls, patients with no admissions correctly return zero.

Q7: [JuanMa*]*

List all patients who entered the hospital through the Emergency Department and were later transferred to an ICU or CCU during the same admission.

Query:

Figure 15: Query 7

```
SELECT DISTINCT p.name
FROM patient p
JOIN admission a ON a.patient_id = p.patient_id
JOIN icu_stay i ON i.admission_id = a.admission_id
WHERE a.admission_source = 'Emergency Room'
  AND i.unit_type IN ('ICU', 'CCU')
  AND i.entry_date_time > a.admission_date_time;
```

Output:

Figure 16:Output Query 7

	name character varying (100)
1	Amelie Gagnon
2	Rachel Williams
3	Rachel Smith
4	Michael Roy

Logic: joins patient, admission, and icu_stay, keeping only admissions whose source is the Emergency Room that also have an ICU or CCU stay beginning after the admission time, so it captures patients who arrived through the ED and were later transferred into critical care during that same admission.

Q8:

Retrieve the most frequently assigned primary ICD-9 diagnosis among admissions that included at least one ICU or CCU stay. Display the ICD-9 code, its official description, and the number of times it was assigned.

Query:

Figure 17: Query 8

```
SELECT
    d.icd9_code,
    dict.official_diagnosis_title,
    COUNT(*) AS times_assigned
FROM diagnosis d
JOIN icd9_dict dict ON dict.icd9_code = d.icd9_code
WHERE d.primary_diagnosis = TRUE
  AND EXISTS (
      SELECT 1 FROM icu_stay i
      WHERE i.admission_id = d.admission_id
        AND i.unit_type IN ('ICU', 'CCU')
  )
GROUP BY d.icd9_code, dict.official_diagnosis_title
ORDER BY times_assigned DESC
FETCH FIRST 1 ROW ONLY;
```

Output:

Figure 18: Output Query 8

	icd9_code character varying (10)	official_diagnosis_title character varying (255)	times_assigned bigint
1	250.00	Diabetes mellitus without complicati…	2

Logic: joins the diagnosis table to icd9_dict on the ICD-9 code to attach each diagnosis's official title, then keeps only primary diagnoses whose admission included at least one ICU or CCU stay (checked with an EXISTS subquery), and groups by code to count how often each was assigned. Ordering by that count descending and taking the first row returns the single most frequent primary diagnosis.

Q9: [*Mohammad]*

Calculate the **average length of stay** for each critical care unit type, such as MICU, SICU, ICU, and CCU. Display the unit type and its average length of stay.

Query:

Figure 19: Query 9

```
SELECT
    unit_type,
    AVG(exit_date_time - entry_date_time) AS average_length_of_stay_days
FROM icu_stay
WHERE exit_date_time IS NOT NULL
GROUP BY unit_type
ORDER BY unit_type;
```

Output:

Figure 20: Output Query 9

Data Output | Messages | Notifications

	unit_type character varying (50)	average_length_of_stay_days interval
1	CCU	2 days 08:30:00
2	ICU	2 days 10:30:00
3	MICU	3 days 14:30:00
4	SICU	4 days 31:25:00

Logic: The query calculates each completed critical care stay by subtracting the entry timestamp from the exit timestamp. It groups the stays by unit type and uses AVG to calculate the average length of stay for each unit. Stays without an exit time are excluded because they are not completed.

Q10: [*Mohammad]*

List all patients who underwent a **surgical procedure before entering an ICU or CCU** during the same admission. Display the patient's name, procedure date and time, critical care unit type, and unit entry time.

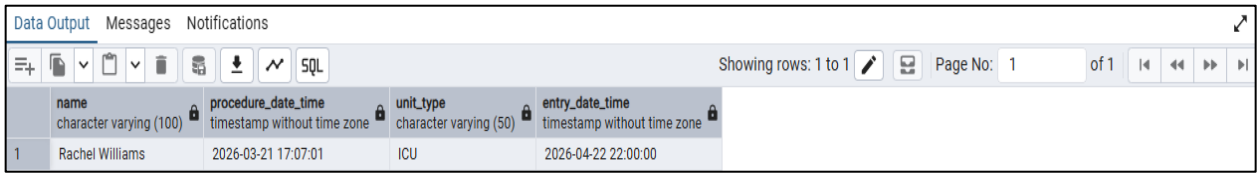
Query:

Figure 21: Query 10

```
SELECT
    p.name,
    sp.procedure_date_time,
    i.unit_type,
    i.entry_date_time
FROM patient p
JOIN admission a
    ON p.patient_id = a.patient_id
JOIN surgical_procedure sp
    ON a.admission_id = sp.admission_id
JOIN icu_stay i
    ON a.admission_id = i.admission_id
WHERE i.unit_type IN ('ICU', 'CCU')
  AND sp.procedure_date_time < i.entry_date_time
ORDER BY p.name;
```

Output:

Figure 22: Output Query 10



	name character varying (100)	procedure_date_time timestamp without time zone	unit_type character varying (50)	entry_date_time timestamp without time zone
1	Rachel Williams	2026-03-21 17:07:01	ICU	2026-04-22 22:00:00

Logic: The query joins patients with their admissions, surgical procedures, and critical care stays. It keeps only ICU or CCU stays and checks that the surgical procedure occurred before the unit entry time during the same admission. The result displays the patient, procedure time, unit type, and ICU/CCU entry time.

Q11: [*Mohammad]*

Retrieve the name of each patient and the **total number of radiology examinations** received across all admissions. Including patients who have not received any radiology examinations and display zero for their examination count.

Query:

Figure 23: Query 11

```
SELECT
    p.name,
    COUNT(re.examination_id) AS total_radiology_examinations
FROM patient p
LEFT JOIN diagnostic_order d
    ON p.patient_id = d.patient_id
LEFT JOIN radiology_examination re
    ON d.order_id = re.order_id
GROUP BY p.patient_id, p.name
ORDER BY p.name;
```

Output:

Figure 24: Output Query 11

Data Output | Messages | Notifications

	col_1 character varying (100)	total_radiology_examinations bigint	col_2 character varying (100)	total_radiology_examinations bigint
1	Amelie Gagnon	1	Jean-Francois Roy	0
2	Amelie Roy	0	Jessica Gagnon	0
3	Amelie Taylor	0	Jessica Taylor	0
4	Amelie Tremblay	3	Jessica Tremblay	0
5	Chloe Boucher	0	Jessica Tremblay	1
6	Daniel Charette	2	John Tremblay	0
7	Daniel Charette	0	John Williams	0
8	Daniel Gagnon	0	Marc-Andre Tremblay	0
9	Daniel Gagnon	0	Michael Roy	1
10	Daniel Gagnon	0	Michael Smith	0
11	Daniel Miller	0	Michael Taylor	2
12	Daniel Smith	0	Rachel Smith	1
13	Daniel Tremblay	0	Rachel Williams	1
14	David Charette	0	Robert Charette	2
15	David Gagnon	0	Robert Gagnon	0
16	David Jones	0	Robert Jones	0
17	David Miller	0	Robert Miller	1
18	David Williams	1	Sarah Charette	0
19	Elena Petrova	0	Sarah Miller	0
20	Emily Jones	0	Sarah Smith	4
21	Emily Jones	2	Sarah Taylor	0
22	Emily Roy	1	Sarah Tola	0
23	Emily Taylor	0	[null]	[null]

Logic: The query starts with all patients and uses LEFT JOIN to connect them to their admissions, diagnostic orders, and radiology examinations. It groups the results by patients and counts the examinations for each patient. The LEFT JOIN ensures that patients with no radiology examinations are still included with a count of zero.

Q12: [*Mohammad]*

Find all patients whose **discharge-summary clinical note** contains the keyword “recovery”. The search should be case-insensitive.

Query:

Figure 25: Query 12

```
SELECT DISTINCT
    p.name
FROM patient p
JOIN admission a
    ON p.patient_id = a.patient_id
JOIN clinical_note cn
    ON a.admission_id = cn.admission_id
WHERE cn.type = 'Discharge Summary'
  AND cn.contents ILIKE '%recovery%'
ORDER BY p.name;
```

Output:

Figure 26: Output Query 12

Data Output | Messages | Notifications

	name character varying (100)
1	Amelie Gagnon
2	Amelie Tremblay
3	Emily Jones
4	Jessica Tremblay
5	John Tremblay
6	Michael Roy
7	Rachel Smith
8	Rachel Williams
9	Robert Charette
10	Robert Jones
11	Sarah Charette
12	Sarah Smith

Total rows: 12 Query complete 00:00:00.338

Logic: The query joins patients with their admissions and clinical notes, then keeps only discharge summary notes. LOWER converts the note contents to lowercase and LIKE '%recovery%' searches for the word “recovery” regardless of capitalization. DISTINCT ensures that each patient appears only once in the result.

Q13: [Laurent]

List all admissions in which the patient had at least one radiology examination but

had no recorded ICU or CCU stay.

Query:

Figure 27: Query 13

```sql
SELECT DISTINCT
  patient.name,
  admission.patient_id,
  admission.admission_id,
  radiology_examination.examination_id,
  icu_stay.unit_stay_id
FROM admission
-- LEFT JOINs to produce a table with all icu_stays + radiology_examinations according to admission
-- (Just JOIN would reject null columns but we want to keep to filter out later)
LEFT JOIN icu_stay ON admission.admission_id = icu_stay.admission_id
LEFT JOIN diagnostic_order ON admission.admission_id = diagnostic_order.admission_id
LEFT JOIN radiology_examination ON diagnostic_order.order_id = radiology_examination.order_id
LEFT JOIN patient ON admission.patient_id = patient.patient_id
-- Filter out admissions without radiology examinations and that has some icu_stay
WHERE icu_stay.admission_id IS NULL
  AND radiology_examination.examination_id IS NOT NULL
ORDER BY admission.patient_id;
```

Output:

Figure 28: Output Query 13

	name character varying (100)	patient_id integer	admission_id integer	examination_id integer	unit_stay_id integer
1	Rachel Williams	4	16	6	[null]
2	Emily Roy	6	45	18	[null]
3	Jessica Tremblay	9	8	2	[null]
4	Robert Miller	10	50	22	[null]
5	David Williams	14	33	14	[null]
6	Amelie Tremblay	15	22	9	[null]
7	Amelie Tremblay	15	31	12	[null]
8	Amelie Tremblay	15	43	17	[null]
9	Robert Charette	20	19	7	[null]
10	Robert Charette	20	23	10	[null]
11	Emily Jones	24	46	19	[null]
12	Emily Jones	24	49	21	[null]
13	Amelie Gagnon	28	42	16	[null]
14	Sarah Smith	35	21	8	[null]
15	Sarah Smith	35	25	11	[null]
16	Sarah Smith	35	32	13	[null]
17	Sarah Smith	35	48	20	[null]
18	Daniel Charette	39	38	15	[null]

Explanation: *Logic explanation in the query comments*

Q14: [Laurent]

Retrieve the patient or patients with the longest completed hospital stay, calculated from admission time to discharge time. If multiple patients have the same maximum length of stay, display all of them.

Query:

Figure 29: Query 14

```sql
SELECT
  patient.patient_id,
  patient.name,
  admission.admission_id,
  admission.admission_date_time,
  admission.discharge_date_time,
  (admission.discharge_date_time - admission.admission_date_time) AS length_of_stay
FROM admission
-- JOIN for admission + patient table
JOIN patient ON admission.patient_id = patient.patient_id
-- Filter for maximum length of stay
WHERE (admission.discharge_date_time - admission.admission_date_time) = (
    SELECT MAX(discharge_date_time - admission_date_time) FROM admission);
```

Output:

Figure 30: Output Query 14

	patient_id integer	name character varying (100)	admission_id integer	admission_date_time timestamp without time zone	discharge_date_time timestamp without time zone	length_of_stay interval
1	15	Amelie Tremblay	22	2026-02-12 15:26:47	2026-02-26 15:26:47	14 days
2	20	Robert Charette	23	2025-12-13 21:36:47	2025-12-27 21:36:47	14 days
3	33	Emily Jones	36	2026-08-16 21:00:47	2026-08-30 21:00:47	14 days

Explanation: *Logic explanation in the query comments*

Q15: [Laurent]

Count the total number of transfers between hospital units for each patient. The patient's first placement is not considered a transfer; each subsequent unit stay within the same admission represents one transfer.

Query:

Figure 31: Query 15

```
SELECT
  patient.name,
  admission.patient_id,
  icu_stay.admission_id,
  COUNT(*) - 1 AS total_transfers -- First transfer doesn't count
FROM icu_stay
-- JOIN for icu_stay + admission + patient table
JOIN admission ON icu_stay.admission_id = admission.admission_id
JOIN patient ON admission.patient_id = patient.patient_id
GROUP BY patient.name, admission.patient_id, icu_stay.admission_id;
```

Output

Figure 32: Output Query 15

	name character varying (100)	patient_id integer	admission_id integer	total_transfers bigint
1	Michael Taylor	13	4	1
2	Amelie Gagnon	28	20	1
3	Sarah Smith	35	11	0
4	Daniel Charette	39	10	0
5	Rachel Williams	4	7	0
6	Michael Roy	30	12	1
7	Daniel Charette	39	13	0
8	Michael Smith	32	6	0
9	Amelie Gagnon	28	17	0
10	Rachel Smith	27	9	1
11	Michael Roy	30	18	0

Explanation: *Logic explanation in the query comments*

Q16: [Laurent]

List all patients who stayed in more than one type of critical care unit during the same admission. For example, a patient may have been transferred from an ICU to a CCU.

Query:

Figure 33: Query 16

```
SELECT
  patient.patient_id,
  patient.name,
  admission.admission_id,
  COUNT (DISTINCT icu_stay.unit_type) AS distinct_unit_types
FROM admission
-- JOIN of patient + admission + icu_stay to get a table
JOIN patient ON admission.patient_id = patient.patient_id
JOIN icu_stay ON admission.admission_id = icu_stay.admission_id
GROUP BY patient.patient_id, patient.name, admission.admission_id
-- Filter out with distinct unit types greater than 1
HAVING COUNT (DISTINCT icu_stay.unit_type) > 1;
```

Output:

Figure 34: Output Query 16

	patient_id integer	name character varying (100)	admission_id integer	distinct_unit_types bigint
1	13	Michael Taylor	4	2
2	27	Rachel Smith	9	2
3	28	Amelie Gagnon	20	2
4	30	Michael Roy	12	2

Explanation: *Logic explanation in the query comments*

Q17: [Freddy Mvemba]

Find all admissions during which the patient received **more than one distinct ICD-9 diagnosis**. Display the patient's name, admission identifier, and number of distinct ICD-9 codes.

Query :

Figure 35: Query 17

```
SELECT p.name,
       a.admission_id,
       COUNT(DISTINCT d.icd9_code) AS num_distinct_diagnoses
FROM patient P
JOIN admission a ON p.patient_id = a.patient_id
JOIN diagnosis d ON a.admission_id = d.admission_id
GROUP BY p.name, a.admission_id
HAVING COUNT(DISTINCT d.icd9_code) > 1;
```

Output

Figure 36: Output Query 17

	name character varying (100)	admission_id integer	num_distinct_diagnoses bigint
1	Amelie Gagnon	2	2
2	Amelie Taylor	3	2
3	Amelie Tremblay	31	2
4	Daniel Charette	38	2
5	David Charette	47	2
6	David Williams	41	2
7	Emily Jones	49	2
8	Emily Roy	45	2
9	Jessica Taylor	37	2
10	John Tremblay	14	2
11	Michael Roy	12	2
12	Michael Roy	18	2
13	Michael Taylor	15	2
14	Robert Miller	24	2
15	Robert Miller	50	2
16	Sarah Miller	28	2
17	Sarah Smith	27	2
18	Sarah Smith	32	2

Logic: Group every admission's diagnoses together, count how many distinct ICD-9 codes appear, and keep only the admissions where that count exceeds 1.

Q18 – [Freddy Mvemba]

Retrieve the **latest clinical note for each patient**. Display the patient's name, note type, note creation date and time, and note contents.

Query:

Figure 37: Query 18

```
SELECT DISTINCT ON (p.patient_id)
       p.name, cn.contents, cn.creation_date, cn.patient_symptoms
FROM patient p
JOIN admission a ON a.patient_id = p.patient_id
JOIN clinical_note cn ON cn.admission_id = a.admission_id
ORDER BY p.patient_id, cn.creation_date DESC;
```

Output:

Figure 38: Output Query 18

Showing rows: 1 to 20 | Page No: 1 of 1

	name character varying (100)	contents text	creation_date timestamp without time zone	patient_symptoms text
1	Rachel Williams	Progress note: Patient reports feeling better today. Vital signs are within normal parameters, continuing curr...	2026-08-14 14:00:00	Persistent productive cough, high fever, chills, and pleuritic chest pa
2	Jessica Tremblay	Patient comfortable, resting in bed. No acute distress or new complaints reported during rounds.	2026-05-12 13:00:00	Acute abdominal pain localized in the right lower quadrant, nausea,
3	Sarah Charette	Headache has subsided completely and cerebrospinal fluid cultures returned negative. Patient is safe for ho...	2026-08-10 21:00:00	Persistent productive cough, high fever, chills, and pleuritic chest pa
4	Michael Taylor	Progress note: Patient reports feeling better today. Vital signs are within normal parameters, continuing curr...	2026-09-10 19:00:00	Dizziness, palpitations, mild shortness of breath, and fatigue.
5	David Williams	Consultation completed. Recommended monitoring vital signs and checking routine lab work in 24 hours.	2026-05-23 12:00:00	Dizziness, palpitations, mild shortness of breath, and fatigue.
6	Amelie Tremblay	Neurological deficits have significantly improved with regular physical therapy. Patient is discharged to a reh...	2026-04-22 05:00:00	Persistent productive cough, high fever, chills, and pleuritic chest pa
7	Robert Charette	Pain is well-controlled with oral medications. Patient is ambulatory and ready to continue their recovery journ...	2026-01-12 23:00:00	Persistent productive cough, high fever, chills, and pleuritic chest pa
8	Emily Jones	Reviewed morning laboratory results and adjusted medication dosages accordingly. Will re-evaluate tomorro...	2026-06-25 10:00:00	Generalized fatigue, unexplained weight loss, increased thirst, and f
9	Sarah Miller	Emergency evaluation completed. Patient stabilized and currently awaiting inpatient bed assignment.	2026-04-28 11:00:00	Acute abdominal pain localized in the right lower quadrant, nausea,
10	Rachel Smith	Patient has shown excellent progress following treatment. Vital signs are completely stable, and the patient i...	2026-09-27 14:00:00	Acute abdominal pain localized in the right lower quadrant, nausea,
11	Amelie Gagnon	Emergency evaluation completed. Patient stabilized and currently awaiting inpatient bed assignment.	2026-08-26 14:00:00	Sudden onset of weakness on the left side of the body, slurred spee
12	Michael Roy	Pain is well-controlled with oral medications. Patient is ambulatory and ready to continue their recovery journ...	2026-08-22 20:00:00	Acute abdominal pain localized in the right lower quadrant, nausea,
13	Robert Jones	Neurological deficits have significantly improved with regular physical therapy. Patient is discharged to a reh...	2026-06-17 12:00:00	Severe chest pain radiating to the left arm, shortness of breath, and
14	Michael Smith	Progress note: Patient reports feeling better today. Vital signs are within normal parameters, continuing curr...	2026-09-25 11:00:00	Persistent productive cough, high fever, chills, and pleuritic chest pa
15	Emily Jones	Reviewed morning laboratory results and adjusted medication dosages accordingly. Will re-evaluate tomorro...	2026-03-26 11:00:00	Severe chest pain radiating to the left arm, shortness of breath, and
16	Daniel Tremblay	Patient comfortable, resting in bed. No acute distress or new complaints reported during rounds.	2026-02-05 16:00:00	Severe chest pain radiating to the left arm, shortness of breath, and
17	Sarah Smith	Normal sinus rhythm has been successfully restored and maintained. Discharge instructions provided regard...	2026-06-10 19:00:00	Severe throbbing headache, photophobia, neck stiffness, and nause
18	John Tremblay	Surgical incision is healing well with no signs of erythema or infection. The post-operative recovery process i...	2026-06-29 21:00:00	Generalized fatigue, unexplained weight loss, increased thirst, and f
19	Amelie Taylor	Progress note: Patient reports feeling better today. Vital signs are within normal parameters, continuing curr...	2026-07-17 09:00:00	Dizziness, palpitations, mild shortness of breath, and fatigue.
20	Daniel Charette	Patient comfortable, resting in bed. No acute distress or new complaints reported during rounds.	2026-05-26 19:00:00	Dizziness, palpitations, mild shortness of breath, and fatigue.

Total rows: 20 | Query complete 00:00:00.286 | CRLF | Ln 234, Col 30

Main logic: For each patient, pick the single clinical note with the latest creation_date_time across all their admissions. Join patient → admission → clinical_note, then keep only the top row per patient (DISTINCT ON (patient_id) or ROW_NUMBER () = 1).

Q19 – [Freddy Mvemba]

List all admissions where the patient's discharge status or discharge condition indicates that the patient **died during the hospital stay**. Display the patient's name, admission date, recorded date of death, and associated primary ICD-9 diagnosis.

Query:

Figure 39: Query 19

```
-- Q19: List all admissions where the patient's discharge status or discharge condition indicates
-- that the patient died during the hospital stay. Display the patient name, admission
-- date, recorded date of death, and associated primary ICD-9 diagnosis.
SELECT
    p.name ,
    a.admission_id,
    d.discharge_date_time,
    dg.icd9_code
FROM admission a
JOIN patient           p  ON p.patient_id       = a.patient_id
JOIN discharge_record  d  ON d.admission_id     = a.admission_id
LEFT JOIN diagnosis    dg ON dg.admission_id    = a.admission_id
                          AND dg.primary_diagnosis = TRUE
WHERE d.condition_at_discharge = 'Deceased'
ORDER BY d.discharge_date_time;
```

Output:

Figure 40: Output Query 19

Data Output | Messages | Notifications

	name character varying (100)	admission_id integer	discharge_date_time timestamp without time zone	icd9_code character varying (10)
1	Amelie Gagnon	42	2026-03-14 10:00:47	410.9
2	Daniel Charette	10	2026-07-25 10:01:47	401.9
3	Sarah Smith	11	2026-09-07 02:26:47	410.9

Main logic: Find admissions where the patient died during the stay, by filtering discharge_record.condition_at_discharge = 'Deceased'. Join admission → patient → discharge_record, then LEFT JOIN diagnosis (on primary_diagnosis = TRUE) to attach the primary ICD-9 code.

Q20 – [Freddy Mvemba]

Find all patients who underwent a **surgical procedure and a radiology examination on the same calendar day** during the same admission. Display the patient's name, admission identifier, procedure type, radiology examination type, and the common date.
Query:

Figure 41: Query 20

```
SELECT
    p.name,
    a.admission_id,
    sp.procedure_id,
    re.examination_type,
    DATE(sp.procedure_date_time) AS common_date
FROM patient p
JOIN admission a ON a.patient_id = p.patient_id
JOIN surgical_procedure sp ON a.admission_id = sp.admission_id
JOIN diagnostic_order d ON d.admission_id = a.admission_id
JOIN radiology_examination re ON re.order_id = d.order_id
WHERE DATE(sp.procedure_date_time) = DATE(re.examination_date_time)
ORDER BY common_date, p.name;
```

Output:

Figure 42: Output Query 20

Data Output | Messages | Notifications

SQL

	name character varying (100)	admission_id integer	procedure_id integer	examination_type character varying (100)	common_date date
1	Sarah Smith	25	9	Abdominal Ultrasound	2026-02-04
2	Sarah Smith	21	6	CT Scan - Chest	2026-09-12
3	David Williams	33	10	Abdominal Ultrasound	2026-09-27

Logic:
Find patients who had both surgery and a radiology exam on the same calendar day in the same admission. Join patient to admission and to surgical_procedure, then admission to diagnostic_order and to radiology_examination and filter with DATE (surgery_time) = DATE (exam_time).

Table of Figures

Figure 1:ER Diagram 3
Figure 2: Live statics of our database 7
Figure 3: Query 1 9
Figure 4: Output Query 1 9
Figure 5: Query 2 10
Figure 6: Output Query 2 10
Figure 7: Query 3 11
Figure 8: Output Query 3 11
Figure 9: Query 4 12
Figure 10: Output Query 4 12
Figure 11: Query 5 13
Figure 12: Output Query 5 13
Figure 13: Query 6 14
Figure 14: Output Query 6 14
Figure 15: Query 7 15
Figure 16:Output Query 7 15
Figure 17: Query 8 16
Figure 18: Output Query 8 16
Figure 19: Query 9 17
Figure 20: Output Query 9 17
Figure 21: Query 10 18
Figure 22: Output Query 10 18
Figure 23: Query 11 19
Figure 24: Output Query 11 19
Figure 25: Query 12 20
Figure 26: Output Query 12 20
Figure 27: Query 13 21
Figure 28: Output Query 13 21
Figure 29: Query 14 22
Figure 30: Output Query 14 22
Figure 31: Query 15 23
Figure 32: Output Query 15 23
Figure 33: Query 16 24
Figure 34: Output Query 16 24
Figure 35: Query 17 25
Figure 36: Output Query 17 25
Figure 37: Query 18 26
Figure 38: Output Query 18 26
Figure 39: Query 19 27
Figure 40: Output Query 19 27
Figure 41: Query 20 28
Figure 42: Output Query 20 28